Name:	**ID:**
Adrián García Cuéllar	2996024
Roberto Isaac Cabral Sifuentes	2993421
Rafael Alfonso Medellín Guillén	2956234
David Silva Padilla	2958539
Juan Pedro Sánchez Villalobos	2961114
Course: Knowledge Management Systems and Technologies	**Teacher:** Alfredo Paul Aguirre Torres
Date: November 7th, 2026	**Activity:** Activity 3

Universidad Tecmilenio

Description of the approach:
To implement a recommender system based on our documents, we used a pretrained Sentence Transformer, which was used to perform semantic recommendations based on embedding.

First, the system loads the CSV file with all the simulated documents. This document includes IDs, titles, content, and categories. The information is validated to ensure all fields are filled and IDs are unique.

Next, the document's title and content of each document are combined and run through the pretrained model to generate embeddings, numerical representations for the texts. These embeddings are done based on the semantic information and will allow the system to compare documents based on their meaning, rather than the exact words they use.

Once a user submits a text query, the model converts this query into an embedding of its own. This embedding is compared using the cosine similarity with the embeddings of all the registered documents to determine the most similar ones based on this value.

The system returns a list of the documents ranked by the similarity scores. In this case, the top 5 are the ones displayed with all their attributes, as well as the similarity value. It's important to point out that the embeddings of all registered documents are only done the moment the program initializes, avoiding unnecessary repetition every time a user types a query.

Development areas of opportunity (areas for improvement):
If this system were to keep being developed, some of the possible steps or improvements that could be considered include the following:

- **New metadata and management:** To give access to as much information as possible, other attributes could be registered beyond the four that are already present for each document. These could include creation/modification dates, author (person or department), document status (active, archived, under review), etc. Also, there should be a possibility for queries to include these other attributes to provide more specifications and filter out documents that might be irrelevant in the context.

- **Security and data protection:** A system like this should offer additional security measures in a production environment. One of the most relevant would be a login/authentication system to ensure only actual users/employees consult the documentation. Other security measures include a query protection against possible code injections and creating a dedicated repository or database to store the CSV file and avoid unauthorized access.

- **Continuous evaluation:** To ensure the appropriate maintenance of the system, it's important to integrate metrics that allow us to identify the actual relevance of the shown results and ranking quality. As an extra improvement, including a feature that allows actual user feedback would let us understand the result quality and possible errors directly from the final user's experience.

- **Multilingual support:** Since there's no guarantee that the documents will always be fully in English, it might be helpful to research and implement multilingual embedding models to support both documents and queries in other languages.

- **Graphical user interface:** Develop a simple, intuitive interface that allows users to do queries as a replacement for the command line.

Link to the organizational reflection video:
https://youtu.be/jU7VCudr3XQ?si=ASHJuErhrM_GelfB

Link to the Github repository:
https://github.com/Guardian7366/documentVectorized/tree/main